THOX.ai

REPORT

ThoxOS Ginux Edition 0.1.0

Ginux review, ThoxOS on an independent Rust kernel, Docker demo and KnightHub SSD image

Field	Value
Owner	Odette Lark, memory-sync node and fleet librarian
Audience	Tommy Xaypanya (CTO), Craig Ross (CEO)
Date	2026-10-29
Version	0.1.0
Classification	Internal

Your AI. Your Data. Your Rules.

SUMMARY

ThoxOS now boots on Ginux, Gregory Kurtzer's Rust kernel that runs Linux programs with no Linux kernel underneath. In the Docker demo a ThoxOS userspace pulled from a private local registry reached systemd multi-user and the login prompt, and the ThoxOS shell ran typed commands on the Ginux kernel. An 8 GiB external-SSD image boots a Debian host that runs Ginux on tty1. Real hardware and KVM are the next gate.

Measure	Value	Note
Docker boot gate	Pass	welcome, multi-user, login prompt
OCI pull in Ginux	38.5 s	4 layers, 110 MB, TCG
SSD host checks	6 of 6	QEMU USB-disk boot
Automated tests	80	pytest; shellcheck clean

01 What shipped

Branch ginux-edition-0.1.0 of ttracx/thoxos-ginux, directory ginux-edition/. It sits beside, not over, the record a ChatGPT/Codex session pushed the same evening.

Layer	What it does
AI	thox ask sends prompts to any OpenAI-compatible server on the host through 10.0.2.2; thox status reports identity, kernel and endpoint. No model ships in the image.
Kernel	Upstream Ginux at 36503bd with three patches: 48-bit IOMMU and firmware discovery, ThoxOS banner, and one per-build registry root accepted only for registry.thoxos.internal.
Userspace	Rocky Linux 10 UBI init (the digest upstream validated) with a ThoxOS identity overlay: os-release, issue, motd, thox, thox-shell.
Backend	Read-only TLS registry and DNS forwarder on host loopback; thoxos-g owns the QEMU profile and drives the Ginux controller.
Frontend	Serial console, browser console (ttyd, one client, generated credential), and tty1 on KnightHub.
Install	8 GiB GPT image (systemd-boot, Debian 13 host, KVM launcher, first-boot growth), Linux writer with guards, Windows writer.

02 Ginux review

Ginux is real and runs Linux userspace. It is also a research prototype: virtio-only devices, one CPU, a 4 GiB identity map, a public development signing key and no security audit.

Finding	Impact	ThoxOS action
QEMU before 9.1 uses a 39-bit IOMMU; Ginux panics	Blocks boot	Patch 0001
QEMU 10.0.13 panics in virtio-net init; 10.2 and 11.1 pass	Blocks boot	Pin QEMU 11.1; doctor flags 10.0
Embedded minimal-system domain panics (stale auxv fixture check)	Upstream defect	Traced; upstream issue queued
Guest RAM above 2.75 GiB panics (identity map)	Limit	Launcher caps at 2816 MiB
Controller seal uses a public test key; no audit	Security	Research-preview label; no guest ports exposed
Console has no line discipline: login and an interactive bash end at the first read	Usability	thox-shell line editor (shell domain)
OCI cache answers by tag: a kept volume boots the previous build's userspace	Stale boot	Volume stamped with the manifest digest; replaced on change

Fit
High demo value, low product value today. THOX device kernels stay Linux; Ginux shows what one expert plus an agent can build.

03 Validation

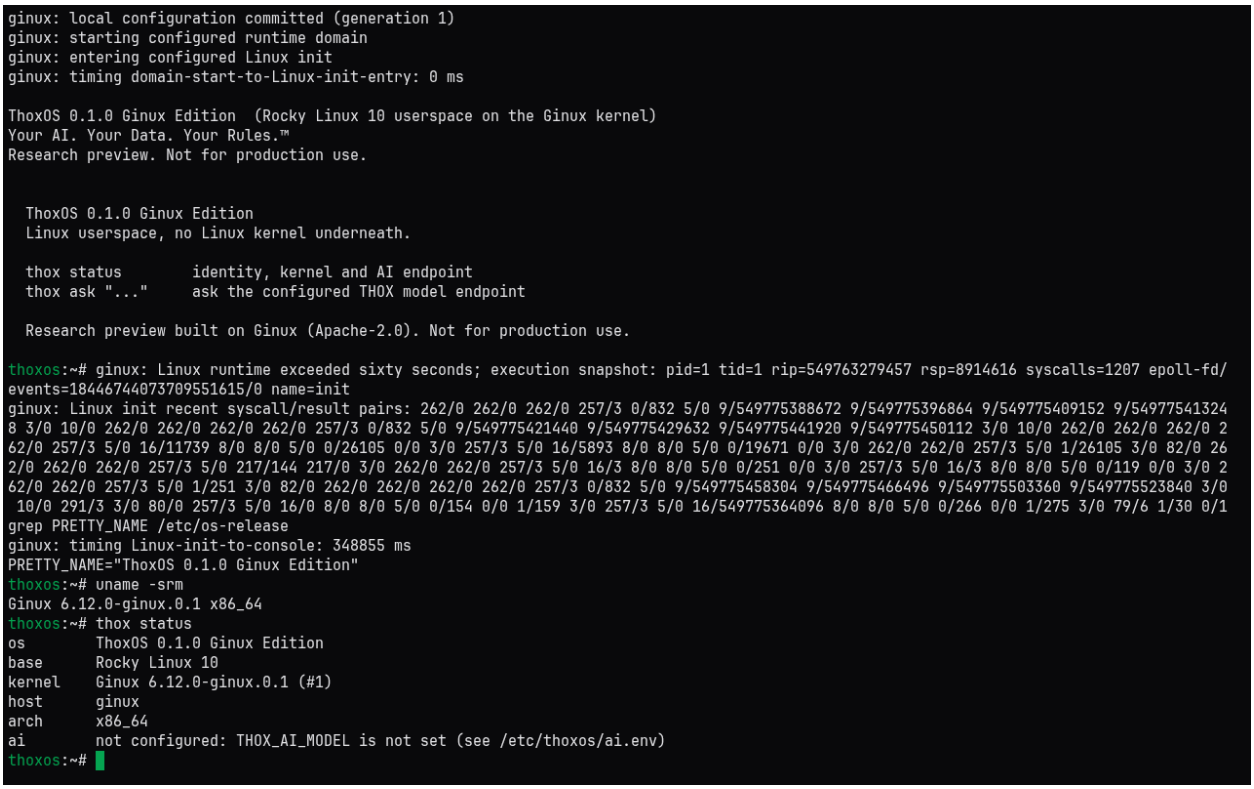
Check	Evidence	Status
Docker selftest, system domain	make selftest exit 0; systemd welcome, multi-user.target, login:	Verified
Shell domain session	uname -srm prints Ginux 6.12.0-ginux.0.1 x86_64; thox status runs	Verified
SSD image boots as USB disk	Kernel, root, growroot 11.5 GiB, registry, DNS, console service; tty1 capture	Verified
Writer guards and write	Refusals for loop, wrong confirmation, THOXOS label; 8 GiB written and verified	Verified
Web console	401 without credential, 200 with; systemd boot and shell-session captures	Verified

Check	Evidence	Status
Stale cache after a rebuild	Failed on a reused volume; after the fix the same volume resets and the shell session passes	Verified
Unit and static checks	80 pytest, shellcheck clean	Verified
KVM boot on KnightHub	Hardware not reachable from this runtime	Pending
Windows writer	No Windows host in this runtime	Pending
Console password login in the domain	Ginux console reads end the session	Blocked

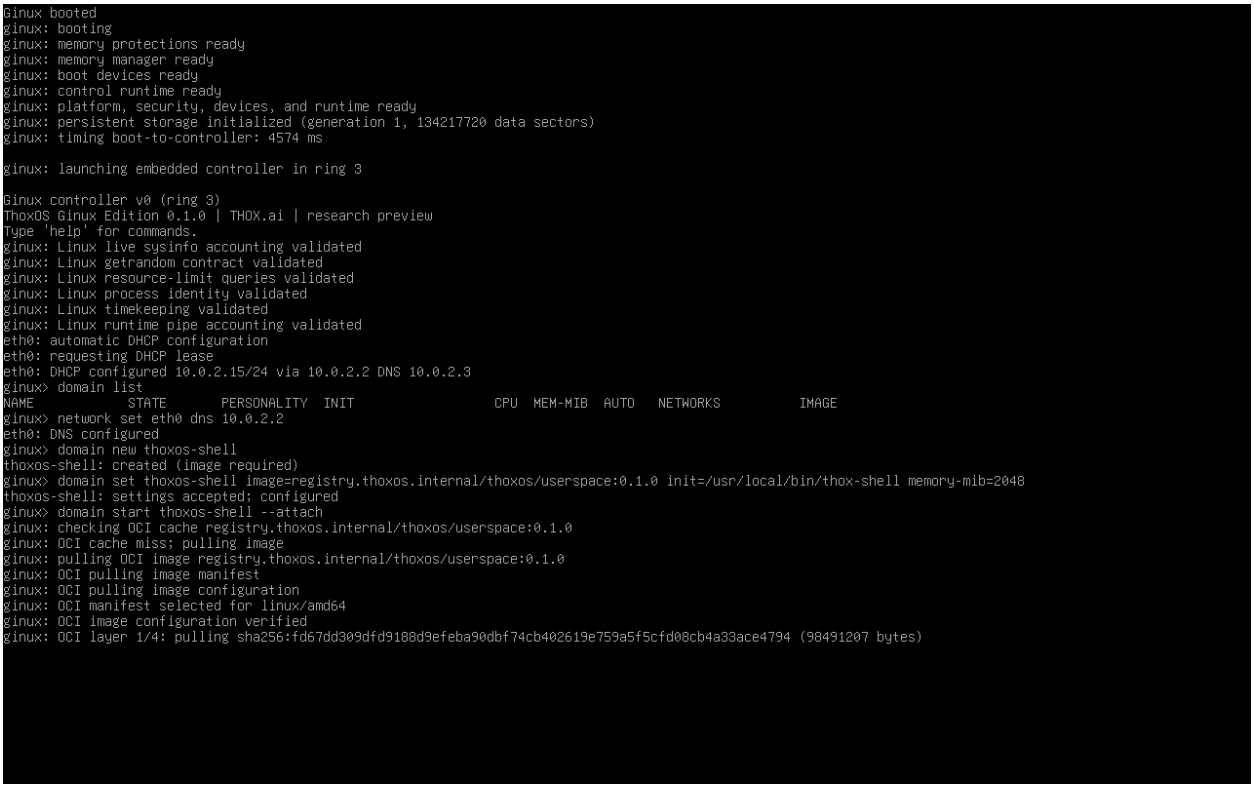
04 Previews



Docker demo, browser console: ThoxOS on the Ginux kernel (QEMU TCG)



Docker demo, browser console in shell mode: commands typed into ThoxOS run on the Ginux kernel (QEMU TCG)



KnightHub SSD image booted as a USB disk in QEMU: tty1 runs Ginux and pulls ThoxOS from the host registry (simulation, not hardware)

05 Install on KnightHub

1. Get the image: the xz file delivered with this report (398 MiB, image SHA-256 6edc11d0…5690), or build it with make ssd-image.
2. Write it to a spare USB SSD: installer/install-to-ssd.sh on Linux takes the .img.xz directly; on Windows decompress with 7-Zip, then use Install-ThoxOSGinux.ps1 or Rufus in DD mode.
3. Disable Secure Boot, open the boot menu, choose the SSD.
4. tty1 shows ThoxOS; Ctrl-Alt-F2 is the host admin login (user thox, new password at first login).

Disk rule
The writer refuses the THOXOS-labeled 500 GB SSD that carries ThoxOS Vibe unless told to erase it. Use a spare SSD.

06 Two records, kept apart

A ChatGPT/Codex session pushed a controller-only research record to main the same evening. It confirms the IOMMU width, the identity-map limit and the embedded-domain panic. This record adds the OCI userspace boot, the Docker run and the SSD image. One summary field there reads Linux domain execution failed; here the OCI domain passes and only the embedded path fails. docs/RECONCILIATION_2026-10-28.md lists every agreement and disagreement for a ruling.

Next steps

1. Write the SSD image to a spare USB SSD and boot KnightHub; capture tty1 and thoxos-g doctor.
2. Rule on the two records and the SSD target (spare disk or ThoxOS Vibe host).
3. Open the upstream pull request for patch 0001 and issues for the QEMU 10.0, auxv and OCI cache findings.